

The background of the slide features a large, faint watermark of the Universidad Mayor de San Andrés (UMSS) logo. It is an inverted triangle with a red border. The top part is white with the letters 'UMSS' in blue. The bottom part is divided into two red triangles meeting at a central point, which contains a stylized grey geometric emblem.

UMSS

Estructuras de Datos Esenciales

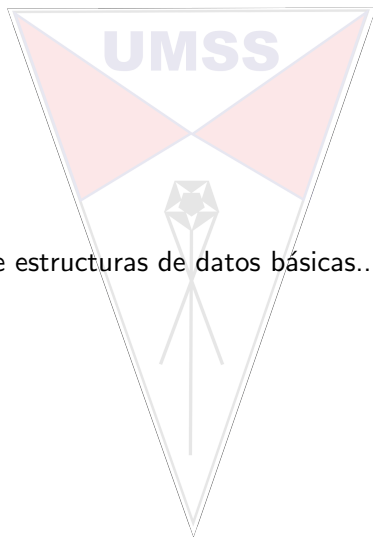
Leticia Blanco

Departamento de Informática - Sistemas
UMSS

4 de septiembre de 2024

Contenido

Revisión rápida de estructuras de datos básicas....



Características

Las estructuras de datos permiten almacenar colecciones de datos:

recorrido

búsqueda

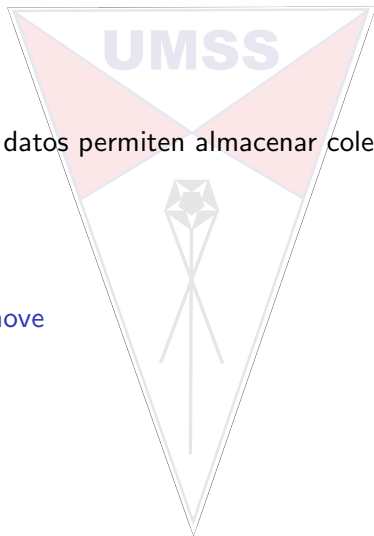
inserciones/put

eliminaciones/remove

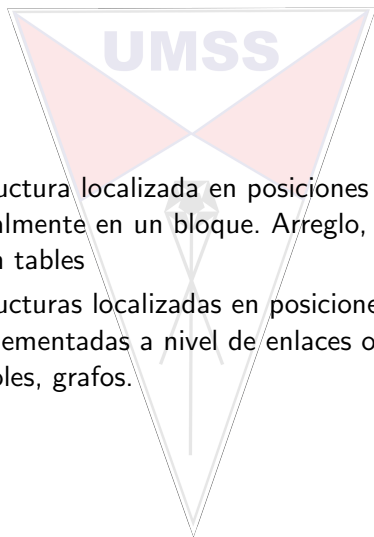
acceso/get

java: *java.util.**;

c++: *stl*



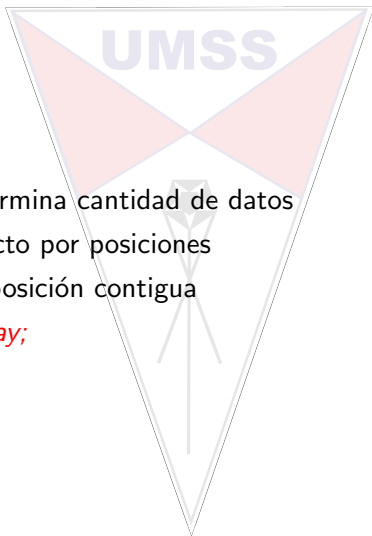
Asignación



Contigua estructura localizada en posiciones contiguas, usualmente en un bloque. Arreglo, matrices, heaps, hash tables

No contigua estructuras localizadas en posiciones no contiguas, implementadas a nivel de enlaces o punteros. Listas, árboles, grafos.

Array - arreglos



Dimensión determina cantidad de datos

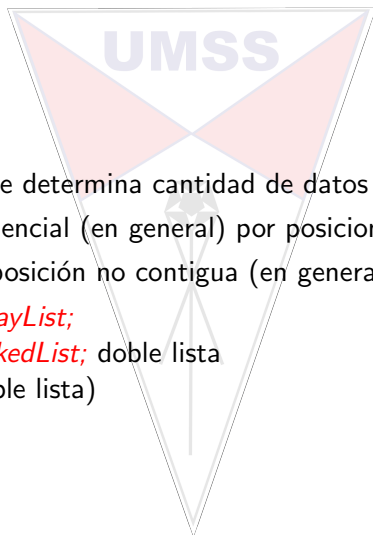
Acceso directo por posiciones

Disposición contigua

java: *java.util.Array;*

c++: *<vector>*

List - listas



Dimensión no se determina cantidad de datos a priori

Acceso secuencial (en general) por posiciones

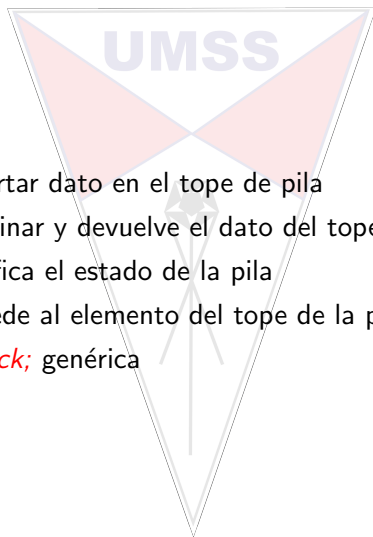
Disposición no contigua (en general)

java: *java.util.ArrayList*;

java: *java.util.LinkedList*; doble lista

c++: *<list>* (doble lista)

Pila - Stack



push insertar dato en el tope de pila

pop eliminar y devuelve el dato del tope de la pila

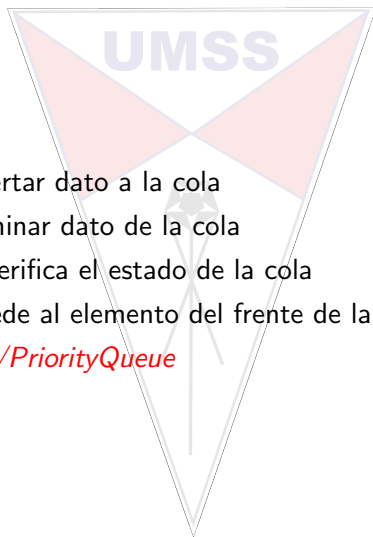
empty verifica el estado de la pila

peek accede al elemento del tope de la pila

java: *java.util.Stack*; genérica

c++: *<stack>*

Queue



`enqueue/add` insertar dato a la cola

`dequeue/poll` eliminar dato de la cola

`empty/isEmpty` verifica el estado de la cola

`front/peek` accede al elemento del frente de la cola

java: *`ArrayDeque/PriorityQueue`*

c++: *`<queue>`*

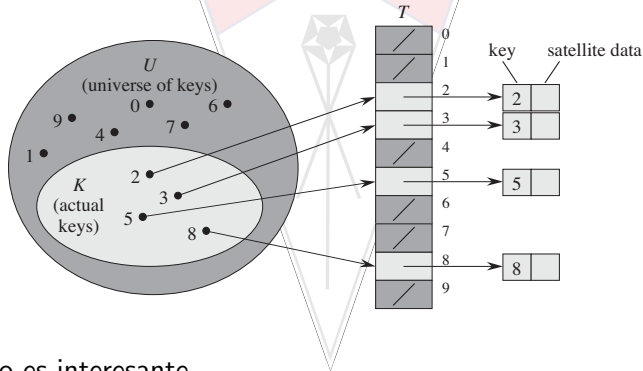
Hashing

Es el mecanismo por el cual se puede encontrar un dato equivalente discreto a un dato dado (usualmente una clave)

$$\textit{hash}(\textit{clave}) = \textit{valorDisc}$$

Direct-address tables

La **función hash** más obvia que existe es que la clave en sí sea su valor discreto (dirección)

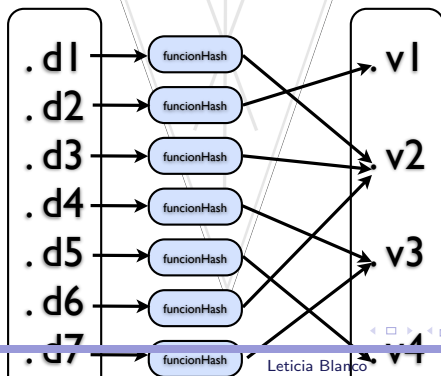


Esto no es interesante

Hash Table

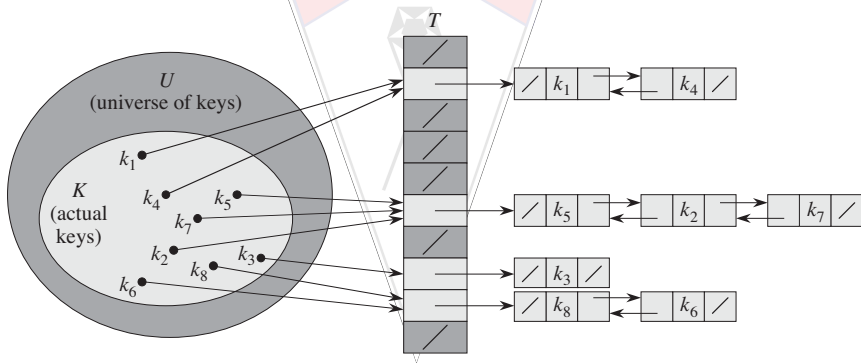
Dispersar un conjunto de datos en porciones discretas - no se tienen tantas direcciones como claves de datos.

Para ello utilizan una **función hash** (función de dispersión), tal que, esta función puede para varios datos distintos, encontrar un mismo equivalente discreto.



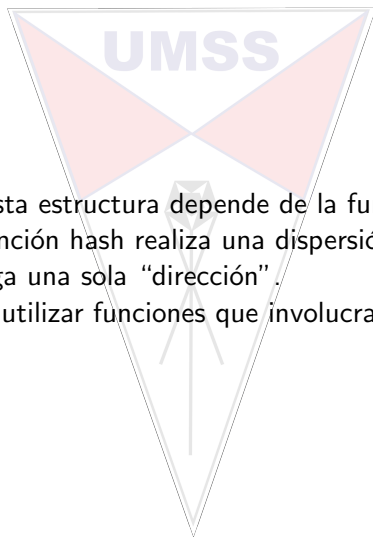
Hash Table

Para evitar las colisiones, se utiliza una estructura adicional que permita albergar los diferentes datos.



Útil para accesos rápidos de información por clave

Hash Table



La eficiencia de esta estructura depende de la función hash, se dice que una buena función hash realiza una dispersión uniforme, es decir no sobrecarga una sola “dirección”. Para ello se suele utilizar funciones que involucran operaciones con números primos.

Dictionary

Un ejemplo de uso de hash table

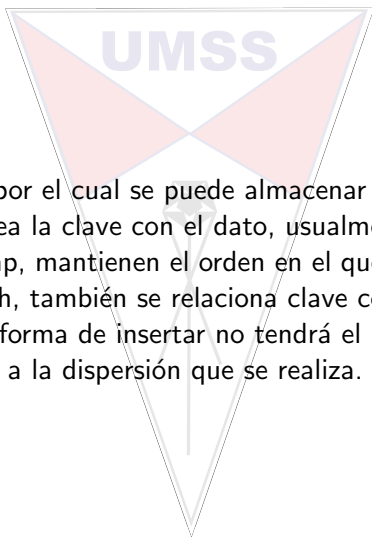
acceso por clave del contenido

search busca un dato por la clave del contenido

insert inserta un dato asociado a una clave

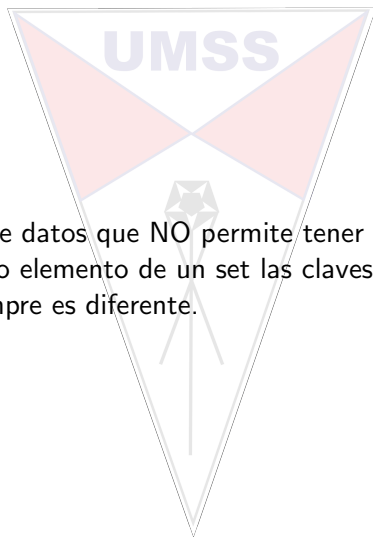
delete devuelve y elimina el dato asociado a una clave

Map



Es el mecanismo por el cual se puede almacenar datos de acceso por la clave. Mapea la clave con el dato, usualmente los datos en una estructura map, mantienen el orden en el que se han insertado. Cuando es un hash, también se relaciona clave con dato, pero probablemente la forma de insertar no tendrá el mismo orden de ubicación. Debido a la dispersión que se realiza.

Set



Es la estructura de datos que NO permite tener datos duplicados, es decir, para todo elemento de un set las claves de cualesquier par de elementos siempre es diferente.

Tree - Arboles

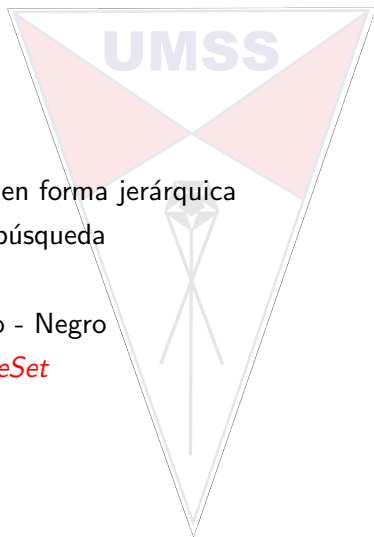
Almacenan datos en forma jerárquica

De búsqueda

AVL

Rojo - Negro

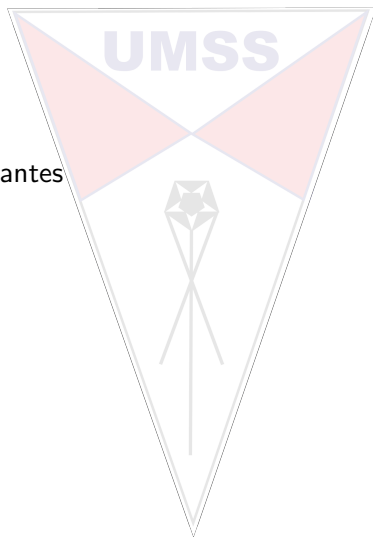
java: *java.util.TreeSet*



Tree - Arboles

Conceptos Importantes

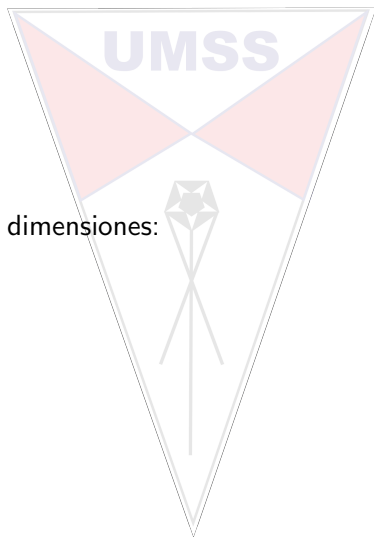
- ▶ Altura
- ▶ Nivel
- ▶ Grado
- ▶ Camino
- ▶ Conectividad



Tree - Árboles

Recorridos en dos dimensiones:

- ▶ Amplitud
- ▶ Profundidad

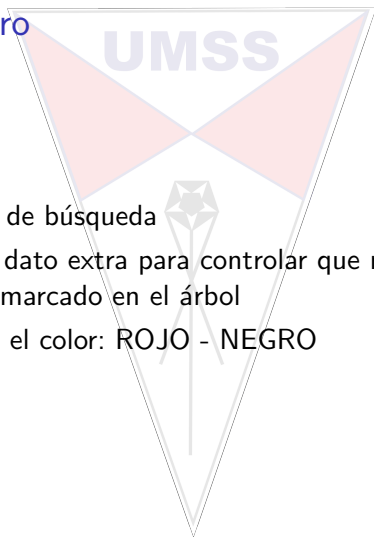


Arbol binario de búsqueda

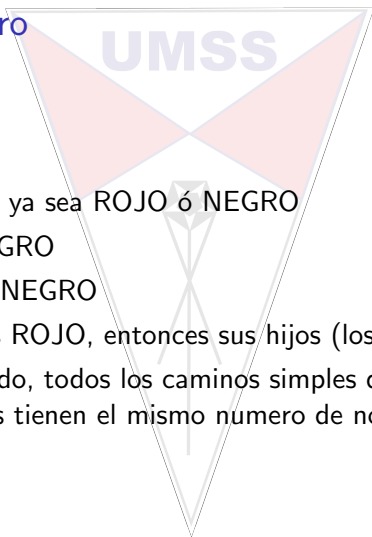
- ▶ Privilegian operaciones de búsqueda
- ▶ Los elementos que almacena deben ser ordenables
- ▶ Se los almacena (inserta) de acuerdo al orden de las claves.
Claves menores a la raiz a la izquierda los demas a la derecha
- ▶ Es deseable que sean balanceados

Arbol Rojo - Negro

- ▶ es un binario de búsqueda
- ▶ almacena un dato extra para controlar que no exista desequilibrio marcado en el árbol
- ▶ dato extra es el color: ROJO - NEGRO



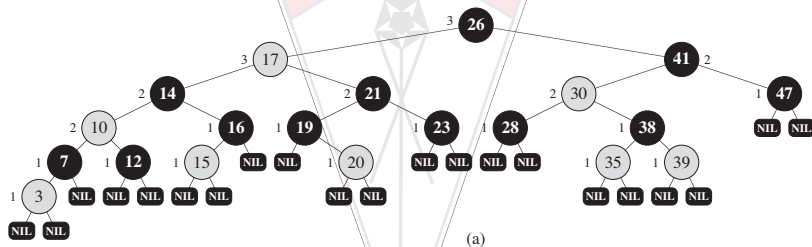
Arbol Rojo - Negro



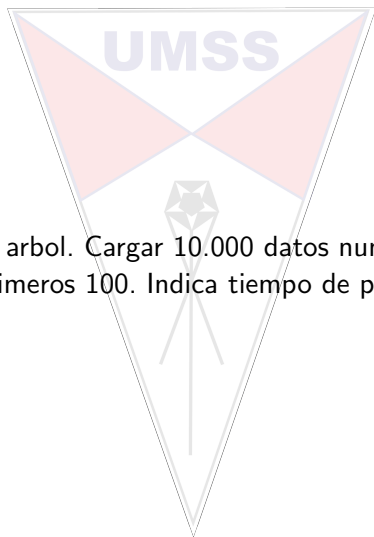
Carateristicas:

- ▶ cada nodo es ya sea ROJO ó NEGRO
- ▶ la raiz es NEGRO
- ▶ cada hoja es NEGRO
- ▶ si un nodo es ROJO, entonces sus hijos (los dos) son NEGRO
- ▶ para todo nodo, todos los caminos simples desde el nodo a los descendientes tienen el mismo numero de nodos NEGRO

Arbol Rojo - Negro



Ejercicio



Probar estructura arbol. Cargar 10.000 datos numéricos aleatorios y encontrar los primeros 100. Indica tiempo de procesamiento.